

UCLA TA1 — AIQ Phase 1 Submission Summary

UCLA TA1 Team (VeriStressGT)

September 29, 2026

Submission. Evaluation card `cards/thrust1_soundness.yaml`, branch `evaluation-card` of <https://github.com/dtroxell19/VeriStressGT>. Setup and run instructions are in `AIQ_README.md` in the repository.

1 Approach

A neural-network verifier either proves that a network is robust, meaning no small perturbation of an input can change its output (UNSAT), or finds a perturbed input that does (a counterexample, SAT). Verifiers are used to certify networks in safety-critical settings, so a verifier bug that wrongly reports “robust” gives false assurance. Catching such bugs requires knowing the right answer, which is what VeriStressGT provides.

VeriStressGT is a benchmark instance generator that tests neural-network verifiers on networks whose robustness answer is known by construction, so a wrong verdict is provably wrong. Existing benchmarks have no ground truth and fall back on majority vote between verifiers.

- **Default Benchmark:** 50 instances (see section Scalability), pre-built in the repository. 24 are UNSAT (i.e. robust) by analytic construction. 26 are SAT (non-robust), each with a counterexample checked in float64 and onnxruntime float32.
- **Verifier pool:** 2 sound in-house controls; 5 real verifiers (α - β -CROWN, PyRAT, nenum, NeuralSAT, Marabou); and 42 deliberately buggy verifiers: 12 mutants of our reference verifier plus 6 output-level faults on each real verifier.
- **Scoring:** every definitive verdict is judged against the known label. The same verdicts are also judged by majority vote, as a baseline.

2 Impact and Comparison with Other Benchmarks

VeriStressGT exposes 39 of the 42 planted bugs; existing benchmarks expose at most 14, and only 6 with certainty. We ran the same verifier pool on two VNN-COMP benchmarks, `mnist_fc` (30 of its 90 instances) and `oval21` (all 30). A planted bug *fires* on a benchmark when it changes at least one of its verifier’s verdicts there.

A bug is “caught by a benchmark’s own labels” when one of that benchmark’s labels shows a verdict to be wrong. VeriStressGT has a known label for every instance. An external benchmark

only has labels an outside party can establish (i.e., SAT where an attack finds a counterexample that checks out numerically). Robustness (UNSAT) cannot be certified externally, so every other instance is unlabelled. On those, the most an external benchmark can do is majority vote between verifiers, as VNN-COMP does, so each benchmark is also scored with its own labels plus majority vote on its unlabelled instances.

Benchmark	Bugs that fire	Caught by own labels	Caught by own labels + majority vote
VeriStressGT	39/42	39/42	39/42
<code>mnist_fc</code> (30/90 subset)	19/42	6/42	14/42
<code>oval21</code>	19/42	2/42	9/42

Majority-vote catches are known to be right here only because we planted the bugs. On a real benchmark nobody can tell whether the flagged verifier or the majority is wrong: on VeriStressGT’s instances, majority vote over the full pool blamed three sound verifiers, including real NeuralSAT, and misjudged 44 of 2,025 verdicts.

The same limit applies to judging individual verdicts. Only a label judges a verdict with certainty; a majority vote only says whether the verdict agrees with the other verifiers.

Benchmark	SAT/UNSAT verdicts	Judged with certainty (labels)	Judged only by majority vote
VeriStressGT	2,025	2,025 (100%)	0 (0%)
<code>mnist_fc</code> (30/90 subset)	953	199 (21%)	754 (79%)
<code>oval21</code>	729	49 (7%)	680 (93%)

- **Bugs that never fire:** 19 of the 42 bugs never change a verdict on either external benchmark, reflecting a lack of benchmark coverage. VeriStressGT exposes 16 of these 19.
- **Real bugs found:** unmodified Marabou 2.0.0 returns false SAT on the provably robust VeriStressGT instances `meap_02`, `meap_03` and `milp_s2_a`, and false UNSAT on `oval21` instance `vb_oval21_0014`, which a certified counterexample shows is SAT.
- **Subset:** the `mnist_fc` row uses 30 of 90 instances for a quicker proof of concept. More instances can only make more bugs fire, so it may understate full `mnist_fc`; `scripts/run_external_comparison.sh` runs all 90.

3 Expected Results and BAA Alignment

Every verifier in the pool gives a verdict (SAT or UNSAT) on every instance, or times out. Because each instance’s correct answer is known, the card checks each verdict and reports two metrics:

- **Scoring accuracy** (`gt_scoring_accuracy` which is the BAA metric with target $\geq 95\%$): of all SAT/UNSAT verdicts, the fraction that the labels judge correctly, i.e. a correct verdict is accepted and a wrong one is rejected.
- **Buggy verifiers caught** (`gt_detection_rate`): of the planted bugs that fired, the fraction shown to be wrong on some instance.

The run should end with **RESULT: VERIFIED**, with both metrics at 1.0 in the top-level `verdict.json`. The card passes only if all three hold:

1. the known labels score 100% of verdicts correctly;
2. they do not falsely flag a sound verifier;
3. they catch at least 75% of the bugs that fired.

Our reference run took 32 minutes on a MacBook Pro (Apple Silicon, 8 GB, CPU only) with MAGNET v0.1.0.

VeriStressGT card results

Metric	Result
Scoring accuracy (BAA metric)	2,025/2,025 verdicts (100%)
correct verdicts wrongly rejected	0
wrong verdicts wrongly accepted	0
Buggy verifiers caught	39/39 fired bugs (100%)
Sound verifiers wrongly flagged	0

4 Scalability

TA2 scales the card with one parameter, `scale: N`, in the card’s `algo_params`. The runner builds a benchmark N times larger with fresh seeds (`thrust1_bench.xN/`) and runs the same verifier pool on it.

- **What grows:** N re-seeded copies of every base network, each keeping its analytic UNSAT certificate, plus N times the random and attack-hard CNN SAT instances.
- **Why results stay consistent:** every label is re-derived from construction and every SAT witness is re-certified, so expected ground-truth accuracy is 100% at any N .
- **Tested:** at `scale: 2`, 98 instances (48 UNSAT, 50 SAT); all 50 witnesses re-certified, ground-truth scoring 173/173, no control flagged (sound controls only, 9 minutes, free size-limited Gurobi license).
- **Needs:** `gurobipy` (installed by the setup script) to build larger benchmarks.

Thrust 2. A separate self-evaluation card (not the submission) predicts per-verifier timeouts: AUC is 0.77–0.92 on held-out data and 0.87–0.96 on fresh networks, all above the 0.7 target.